

CLASSIFICAZIONE AUTOMATICA DI NUMERI

Gruppo di lavoro

- Michele_Loconte, matricola 741221,
m.loconte27@studenti.uniba.it

AA 2025-2026

GITHUB: <https://github.com/lmik00ll/ICON.git>

Introduzione

Il progetto ha come obiettivo la classificazione automatica delle immagini di numeri scritti a mano, in una griglia di 8x8 pixel.(per un totale di 64 feature)

Rientra nell' apprendimento automatico dove vengono analizzati e confrontati i 3 modelli usati: Decision Tree , Support Vector Machine, K-Nearest Neighbors.

Sommario

E' un sistema di apprendimento automatico supervisionato che apprende dal dataset "load_digits" incluso nella libreria scikit-learn che contiene circa 1797 sample di numeri scritti a mano .

Quindi in fase di addestramento ogni matrice 8x8 subisce un appiattimento (flattening) lineare che la trasforma in un vettore unidimensionale a 64 feature continue:

```
dataset = load_digits()
X = dataset['data']    # Matrice di forma (1797, 64)
y = dataset['target']  # Vettore target con classi da 0 a 9
```

Ogni feature corrisponde al valore di intensità di grigio del rispettivo pixel che va da [0,16] nell'intervallo.

ARGOMENTI DI INTERESSE.

Costruzione del ground truth (Preprocessing e Raccolta):

Poiché all'inizio non dispongo di una classificazione manuale esterna per ogni singola immagine, ho usato un dataset validato (load_digits di scikit-learn).

- **1.** Sono partito dal preparare e “pulire” dati (*Data Cleaning*) e della gestione delle feature del dataset standard di scikit-learn (load_digits), strutturando le matrici bitmap originali di 8x8 pixel in array unidimensionali a 64 feature continue e associandole ai target discreti (da 0 a 9).
- **1.2** Raccolta dei dati in un dataset rappresentante il ground truth: organizzazione strutturata delle 1797 istanze complessive suddivise in feature d'ingresso (x) e classi obiettivo (y).
-

Riproduzione del ground truth (Spazio di ricerca, Grafi e Apprendimento supervisionato):

- **2.1 Grafi e Spazio di Ricerca:** Modellazione formale dello spazio delle caratteristiche come uno spazio vettoriale a 64 dimensioni su cui operano i modelli di classificazione, per ciascuna immagine, mappa lo spazio delle istanze come uno spazio vettoriale continuo.
- **2.2 Apprendimento Supervisionato:** Progettazione, addestramento, ottimizzazione degli iperparametri (tramite GridSearchCV) e ho scelto una validazione rigorosa (tramite Nested 5-Fold Cross-Validation) dei tre modelli analizzati:
 - **Albero di Decisione**
 - **Support Vector Machine (SVC)** con kernel linear
 - **K-Nearest Neighbors (KNN).** apprendimento su base Case-Base Reasoning

3. Knowledge Base e Rappresentazione della Conoscenza

Una *knowledge base* (KB) è un sistema strutturato che rappresenta fatti sul mondo, consentendo di effettuare ragionamenti su di essi e di dedurre nuove informazioni o risposte. Nel contesto dei sistemi intelligenti, la KB costituisce l'ambiente volto a facilitare la raccolta, l'organizzazione e la distribuzione della conoscenza. Nel caso di studio, la Knowledge Base viene utilizzata per consentire al sistema di apprendere e all'utente di interrogare il dominio di riferimento tramite la classificazione automatica.

3.1 Gestione della Knowledge Base nel Progetto

A differenza dei sistemi basati su logica proposizionale pura o Prolog, in questo progetto la gestione della Knowledge Base è delegata alla struttura tabulare e vettoriale del dataset `load_digits` integrato nella libreria `scikit-learn` (`main.py`).

Il popolamento della KB avviene caricando in modo strutturato i dati nativi, suddividendoli rigorosamente tra le caratteristiche osservabili e le classi di appartenenza. Nel caso in cui il dataset venisse esteso con nuovi campioni di cifre manoscritte, la KB si aggiornerebbe di conseguenza. Le "regole" di inferenza non sono espresse tramite clausole di Horn o predicati logici tradizionali, ma sono implementate dai modelli di apprendimento supervisionato (*Decision Tree*, *SVM* e *KNN*), i quali elaborano i fatti noti per dedurre la classe di appartenenza di nuove istanze sconosciute.

3.1.1 Fatti (Istanziamento dei Dati)

I fatti rappresentano gli assiomi empirici di partenza presenti nella Knowledge Base, costituiti dai campioni campionati dal dominio reale. Nel sistema, ogni istanza rappresenta una cifra scritta a mano digitalizzata e si compone di relazioni strutturate tra l'immagine e le sue proprietà vettoriali:

1. **`digit_features(sample_id, feature_vector)`** Rappresenta la relazione strutturale tra un'istanza numerica e le sue 64 caratteristiche di input (i pixel della griglia 8x8 normalizzati e appiattiti). *Esempio:* `X = dataset['data']` (vettore di 64 elementi continui nell'intervallo `[0, 16]`).
2. **`digit_target(sample_id, class_label)`** Rappresenta la relazione logica di classificazione tra un'istanza e la sua classe target di appartenenza. *Esempio:* `y = dataset['target']` (valore discreto compreso tra 0 e 9).

4.0 Apprendimento Supervisionato (Classificazione , Modelli di base e valutazione)

L'apprendimento supervisionato è la capacità di un agente di migliorare il suo comportamento tramite l'esperienza passata, migliorandone l'accuratezza e dando all'agente la possibilità di fare più cose. E' un obiettivo di apprendimento automatico caratterizzato da un insieme di features (di input e di target) dove l'obiettivo è predire i valori per le feature target per gli esempi di test non ancora visti.

La base di conoscenza è rappresentata dal dataset standard digits di Scikit-learn, suddiviso in feature (data, array di 64 elementi) e target (target, valori da 0 a 9).

I modelli di ragionamento e apprendimento adottati per risolvere il problema di classificazione multiclasse sono l'Albero di Decisione, la Support Vector Machine (SVC) e il K-Nearest Neighbors (KNN).

La complessità delle regole per l'Albero di Decisione è limitata a una profondità massima pari a (`max_depth=20`), riducendo il rischio di overfitting e limitando la complessità del ragionamento a un massimo di 20 controlli sequenziali sui pixel

La Formalizzazione del Task e lo Spazio delle Feature

La classificazione è multiclasse e richiede che il sistema partizioni lo spazio vettoriale a 64 dimensioni in regioni con confini netti, assegnando ciascuna regione a una specifica cifra. Ogni pixel della griglia funge da feature continua normalizzata in scala di grigi: l'efficacia del classificatore dipende strettamente dalla sua capacità di catturare le correlazioni spaziali e la continuità dei tratti calligrafici distribuiti lungo la matrice.

5.0 Metodologia: Validazione e Prevenzione dell'Overfitting

L'overfitting è il fenomeno in cui un modello si adatta al rumore del training set, perdendo capacità di generalizzazione (basso bias, altissima varianza). Per mitigare il problema e stimare le performance in modo non distorto, il progetto implementa una Nested Cross-Validation.

Per garantire che le stime siano imparziali, ho adottato tecnica della **Nested Cross-Validation** (validazione incrociata nidificata), strutturata su due cicli concentrici .

Livello	Configurazione	Funzione Operativa
Outer CV	5-Fold Cross Validation	Suddivide l'intero dataset in 5 blocchi bilanciati. A rotazione, 4 blocchi (80%) costituiscono il training set esterno e il restante blocco (20%) funge da test set cieco per la stima dell'errore di generalizzazione.
Inner CV	3-Fold Cross Validation	Eseguita internamente su ciascuna porzione dell'80% tramite GridSearchCV. Esplora le combinazioni di parametri addestrando sui due terzi e validando sul terzo rimanente, selezionando i best parameters senza toccare il test esterno.

Questo disaccoppiamento ci ha permesso di prevenire il data leakage.

Il **data leakage** (o "fuga di dati") si verifica quando informazioni provenienti dal set di test o di validazione contaminano in modo improprio il set di addestramento durante la fase di apprendimento di un modello.

STRUMENTI USATI

Libreria scikit-learn (sklearn): Rappresenta il framework fondamentale dell'intero progetto, impiegato per la gestione dei dataset, la modellazione predittiva, l'ottimizzazione degli iperparametri e la validazione statistica:

- `load_digits`: Modulo nativo per il caricamento del dataset benchmark di cifre manoscritte a 64 feature.
- `DecisionTreeClassifier`: Classe per la creazione, l'addestramento e la generazione dell'Albero di Decisione basato su criteri di impurità.
- `SVC` (Support Vector Classifier): Modulo per l'implementazione della Support Vector Machine ottimizzata per la classificazione multiclasse.
- `KNeighborsClassifier`: Classe dedicata all'apprendimento basato su istanze e *Case-Based Reasoning* tramite il calcolo delle distanze metriche.
- `GridSearchCV`: Strumento di ricerca esaustiva su griglia per l'individuazione automatica dei migliori iperparametri tramite validazione incrociata interna.
- `cross_validate` e `cross_val_predict`: Funzioni avanzate per l'esecuzione della valutazione incrociata esterna e per il calcolo dei punteggi di probabilità e funzione decisionale necessari alla generazione delle curve di performance.
- `metrics` (`precision_recall_curve`, `auc`, `label_binarize`): Moduli impiegati per la binarizzazione delle etichette (One-vs-Rest) e il calcolo delle metriche di precisione, richiamo e area sottesa alla curva (AUC).

Libreria pylab (matplotlib environment): Utilizzata per la gestione e il rendering grafico interattivo della curva Precision-Recall (Micro-Averaged), attivando la modalità di disegno in tempo reale (`pl.ion()`) per consentire la visualizzazione dei risultati senza bloccare l'esecuzione dello script principale.

Libreria numpy: Supporto matematico fondamentale per la manipolazione efficiente degli array multidimensionali, il calcolo vettoriale delle predizioni e la determinazione statistica di medie e deviazioni standard (`np.mean`, `np.std`) associate ai report delle metriche sui fold di validazione.

Decisioni di Progetto

Per evitare il bias di campionamento, prevenire il *data leakage* e garantire che le prestazioni e affidabilità delle valutazioni dei modelli da scelte arbitrarie, le predizioni vengono calcolate tramite la tecnica Nested Cross Validation con Outer CV a 5-Fold (per la stima dell' errore) e Inner CV a 3-Fold(per l' ottimizzazione) su tutti i modelli, al fine di garantire dei test su un modello addestrato sull' 80% dei dati.

Nelle fasi di validazione incrociata (cross_validate), ho configurato esplicitamente un dizionario di scoring basato su metriche macro-averaged (precision_macro, recall_macro, f1_macro). Questa scelta progettuale è fondamentale nei problemi di classificazione multiclasse poiché attribuisce lo stesso peso a ciascuna delle 10 cifre (da 0 a 9), impedendo che classi numeriche più semplici o più frequenti falsificano la valutazione globale delle performance (come invece accadeva in precedenza)

I 3 modelli sono configurati tramite i best parameters dati calcolati dalla GridSearchCV come segue:

Support Vector Machine (SVC): Tramite l'ottimizzazione ho potuto confermare l'impiego di un kernel='linear' con parametro di penalizzazione C=1. La scelta di un kernel lineare si fonda sulla natura dello spazio delle feature a 64 dimensioni: trattandosi di un dominio già sufficientemente ampio e continuo, la separazione lineare al margine massimo risulta estremamente efficace, evitando l'overhead computazionale e i rischi di overfitting legati a kernel non lineari più complessi

K-Nearest Neighbors (KNN): L'analisi dei vicini ha identificato in n_neighbors=3 il valore ottimale. Scegliere un valore di k ridotto permette di catturare le variazioni locali e morfologiche tipiche della scrittura manuale dei numeri, evitando l'effetto di appiattimento (alto bias) che si registrerebbe ampliando eccessivamente il raggio di vicinato.

Albero di Decisione (Decision Tree): Per ridurre la tendenza naturale del modello ad andare in overfitting memorizzando il training set, la complessità è stata vincolata impostando una profondità massima max_depth=20 e uno split minimo min_samples_split=2. Questo bilancia la necessità di disporre di un numero sufficiente di livelli logici per discriminare le cifre con la necessità di arrestare la crescita dell'albero prima che intercetti il rumore casuale dei pixel.

Alberi di Decisione (Decision Tree)

Sommario e Fondamenti Teorici

L'Albero di Decisione è un modello simbolico interpretabile in cui ogni nodo interno esegue un test su una caratteristica, ogni ramo uscente corrisponde a un esito del test e ciascuna foglia assegna una classe. La costruzione dell'albero avviene con un algoritmo ricorsivo top-down che seleziona greedy la condizione che minimizza l'impurità dei nodi figli.

Strumenti Utilizzati

Ho stanziato il modello mediante DecisionTreeClassifier del modulo sklearn.tree. L'ottimizzazione è gestita da GridSearchCV.

Decisioni di Progetto e Spazio degli Iperparametri

Gli alberi non vincolati soffrono di un bias di rappresentazione quasi nullo ma di una varianza elevata, arrivando a memorizzare ogni singolo esempio del dataset. Per controllare la complessità e impedire che l'albero scenda in partizioni prive di significatività statistica, ho definito la seguente griglia:

```
param_grid_dt = {'max_depth': [None, 10, 20], 'min_samples_split': [2, 5]}
```

I valori esplorano la possibilità di lasciare l'albero libero (None) o di fissare tetti di profondità a 10 o 20 livelli, combinati con la richiesta di almeno 2 o 5 campioni per tentare una diramazione. La ricerca su griglia ha isolato come configurazione migliore `max_depth=20` e `min_samples_split=2`, confermando la necessità di profondità elevate per discriminare le cifre, pur mantenendo un vincolo rigido oltre il ventesimo livello decisionale.

Valutazione Sperimentale

I report estratti dalle 5 iterazioni di validazione esterna hanno restituito i seguenti punteggi medi e deviazioni standard:

Metrica (Nested 5-Fold CV)	Valore Medio	Deviazione Standard ($\pm \sigma$)
Macro Precision	0.802	± 0.037
Macro Recall	0.794	± 0.043
Macro F1-Score	0.791	± 0.045
Area Under Curve (AUC)	0.794	-

Support Vector Machine (SVC)

Sommario e Fondamenti Teorici

La Support Vector Machine estende i modelli lineari ricercando l'iperpiano di separazione che massimizza il margine geometrico tra le classi. Gli elementi del dataset che si trovano esattamente sul confine del margine prendono il nome di vettori di supporto. La massimizzazione del margine si formula analiticamente come un problema di ottimizzazione convessa quadratica vincolata, garantendo l'assenza di minimi locali e conducendo a un unico ottimo globale.

Strumenti Utilizzati

Per questo modello mi sono affidato alla classe SVC del modulo `sklearn.svm`, gestendo la classificazione multiclasse con architettura One-vs-One (ovvero combinando 45 classificatori binari a coppie).

Decisioni di Progetto e Spazio degli Iperparametri

Nello spazio delle feature a 64 dimensioni, la rappresentazione è già sufficientemente sparsa da consentire un'eccellente separazione lineare diretta. Pertanto, lo spazio di ricerca è stato strutturato definendo:

```
param_grid_svc = {'C': [1, 10, 100], 'kernel': ['linear']}
```

Il parametro di regolarizzazione C agisce come costante di penalità per le violazioni del margine (soft margin): valori più alti impongono margini rigidi penalizzando gli errori, mentre valori moderati consentono una migliore tolleranza alle ambiguità calligrafiche. L'ottimizzazione tramite Inner CV ha confermato come parametro migliore $C=1$ con `kernel='linear'`.

Valutazione Sperimentale

I risultati della validazione nidificata esterna mostrano un netto miglioramento delle prestazioni infatti:

Metrica (Nested 5-Fold CV)	Valore Medio	Deviazione Standard ($\pm \sigma$)
Macro Precision	0.950	± 0.021
Macro Recall	0.948	± 0.021
Macro F1-Score	0.947	± 0.021
Area Under Curve (AUC)	0.961	-

Case-Based Reasoning (K-Nearest Neighbors)

Sommario e Fondamenti Teorici

L'algoritmo K-Nearest Neighbors implementa il paradigma del Case-Based Reasoning (CBR) e dell'apprendimento basato su istanze (instance-based learning). A differenza degli altri approcci che costruiscono un modello compatto offline, il KNN non generalizza nella fase di training: memorizza i casi noti e rimanda l'intero sforzo computazionale alla fase online. Quando giunge una nuova istanza, ne calcola la distanza euclidea rispetto a tutti i punti della base di conoscenza e attribuisce la classe più frequente (moda) tra i k vicini più prossimi.

Strumenti Utilizzati

Il modello adotta la classe `KNeighborsClassifier` del modulo `sklearn.neighbors`, affiancata da `GridSearchCV` per l'ottimizzazione del raggio di vicinato.

Decisioni di Progetto e Spazio degli Iperparametri

Il valore di k governa direttamente la complessità del modello: un k troppo piccolo (es. 1) genera superfici decisionali frammentate e vulnerabili al rumore (alta varianza), mentre un k eccessivamente alto appiattisce le peculiarità locali a favore delle classi maggioritarie (alto bias). La griglia impostata esplora valori dispari per prevenire situazioni di parità:

```
param_grid_knn = {'n_neighbors': [3, 5, 7]}
```

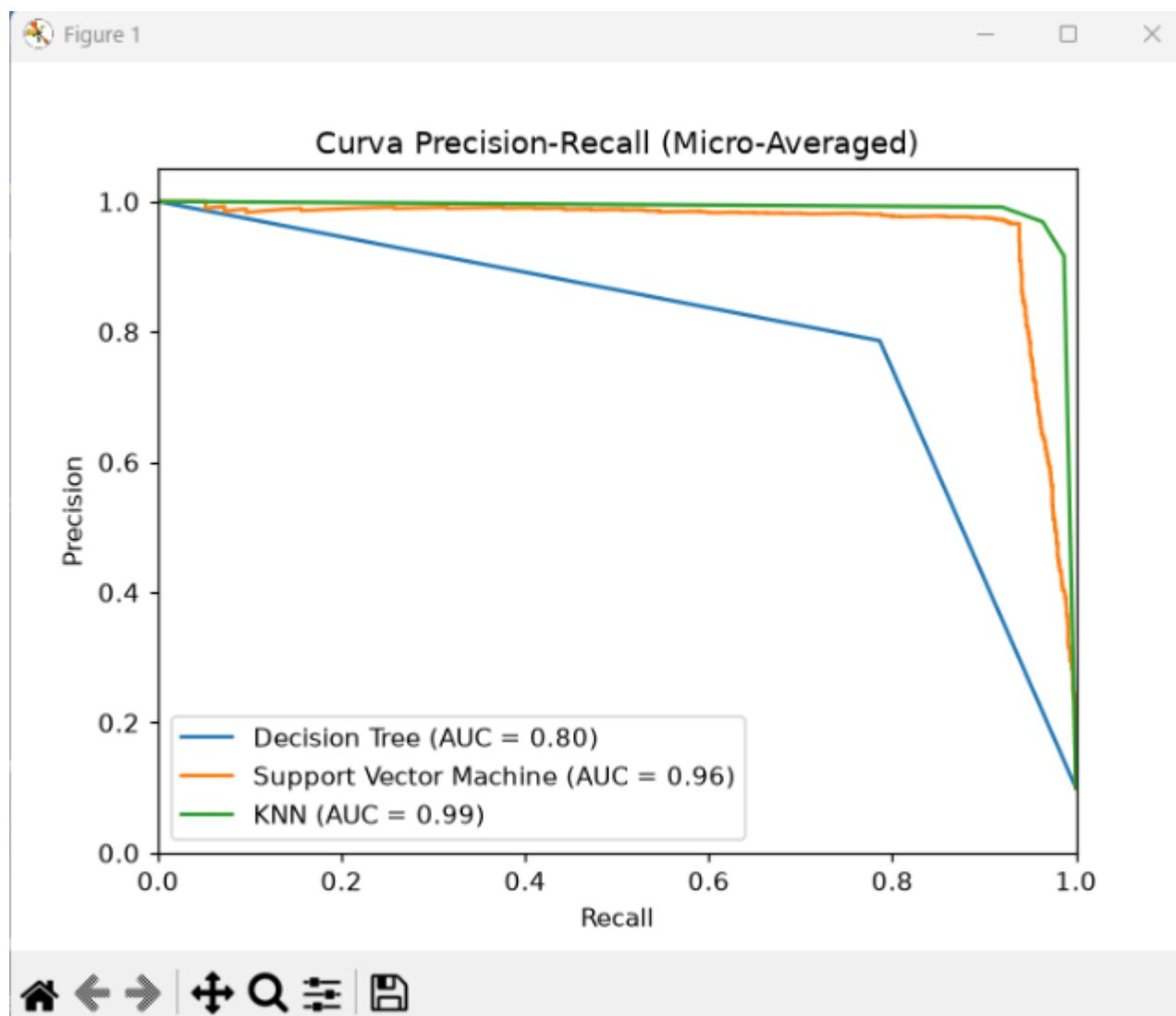
La validazione interna mi ha confermato `n_neighbors=3` come configurazione ottimale, capace di catturare le variazioni locali del tratto numerico senza farsi condizionare da punti isolati.

Valutazione Sperimentale

Il KNN ha ottenuto i valori di accuratezza e separazione più elevati dell'intero studio:

Metrica (Nested 5-Fold CV)	Valore Medio	Deviazione Standard ($\pm \sigma$)
Macro Precision	0.967	± 0.008
Macro Recall	0.965	± 0.009
Macro F1-Score	0.965	± 0.009
Area Under Curve (AUC)	0.988	-

VALUTAZIONE DEI RISULTATI



Il grafico mostra l'andamento tra la **Precision** (asse y: l'accuratezza delle predizioni positive) e la **Recall** (asse x: la capacità del modello di catturare tutte le istanze positive) per ciascun classificatore.

- Più la curva si sposta verso l'angolo in alto a destra, migliori sono le performance del modello.
- L'indicatore sintetico **AUC (Area Under Curve)** riassume l'efficacia complessiva: il *Decision Tree* ottiene un valore di 0.80, la *Support Vector Machine* raggiunge 0.96, mentre il *KNN* domina con un valore quasi perfetto di 0.99. Questo conferma visivamente che KNN e SVC mantengono una precisione elevatissima anche a livelli di richiamo molto alti, superando nettamente l'albero di decisione.

STAMPA DELL' ALBERO

Ho deciso di implementare una ricerca euristica A* per analizzare l'albero generato dall'addestramento. Questo ha permesso di evitare l'esplorazione cieca sull'albero usando una coda di priorità per decidere quale sarà il nodo da esplorare per primo.

Sfrutta l'euristica $h(n) + \text{cost}(n)$ (il costo effettivo del percorso parziale), che permette di trovare il cammino a costo minimo.

Per tanto ho implementato una stampa a video del target, il cammino fatto dall'algoritmo tramite A*, dove 0 è il nodo radice e i valori numerici all'interno della parentesi è il nodo.

Nella seguente immagine si nota che sono stati necessari 5 test prima di ottenere la risposta ottimale.

```
=====
RICERCA A* NELL'ALBERO DI DECISIONE
=====
Classe target cercata: 4
Percorso nodi (radice -> foglia): [0, np.int64(1),
np.int64(2), np.int64(3), np.int64(5), np.int64(11)]
Costo del percorso (profondità): 5
```

STAMPA DEL' OUTPUT

Oltre alla stampa delle tabelle precedentemente mostrate, il programma stampa i valori predetti per verificare che i 3 metodi funzionano.

```
IL VERO NUMERO DA PREDIRE E': 4

=====
NUMERO PREDETTO
=====
Albero decisione: 4
                  SVC: 4
                  Knn: 4
```

Conclusioni

Il progetto mi ha permesso di confrontare 3 modelli, che usano lo stesso dataset per identificare un numero senza l'uso di Computer Vision o Deep learning ma trattando il problema come una classificazione su vettori.

Dai dati ricevuti dai vari test si nota come KNN e SVC abbiano prestazioni migliori rispetto agli alberi di decisioni e infatti restituiscono previsioni più affidabili.

Inizialmente era stata implementata una divisione degli esempi più semplice (2 Fold Cross Validation) che non permetteva un buon addestramento dei modelli. Questo è stato risolto tramite la Outer Fold a 5 e Inner Fold a 3. Quindi ho reputato essenziale fare stampare sempre il numero da predire e le reali predizioni. Nelle svariate prove tutti e tre i metodi sono riusciti a predire il numero, facendomi escludere la possibilità di dover usare Leave-One-Out Cross Validation per migliorare le predizioni.

Tramite la Nested Cross validation ho potuto risolvere il problema dell' overfitting dimostrando che i modelli hanno capacità di generalizzare.

In futuro sarebbe opportuno ampliare i caratteri riconosciuti o utilizzare dataset più complessi confrontando sempre le performance.

Riferimenti Bibliografici

[1] Scikit-learn Developers: *Machine Learning in Python*. Documentazione ufficiale dei moduli `sklearn.svm`, `sklearn.tree`, `sklearn.neighbors` e `sklearn.metrics`.

[2] N. Fanizzi, "Ingegneria della Conoscenza", Dispensa del Corso di Laurea in Informatica.